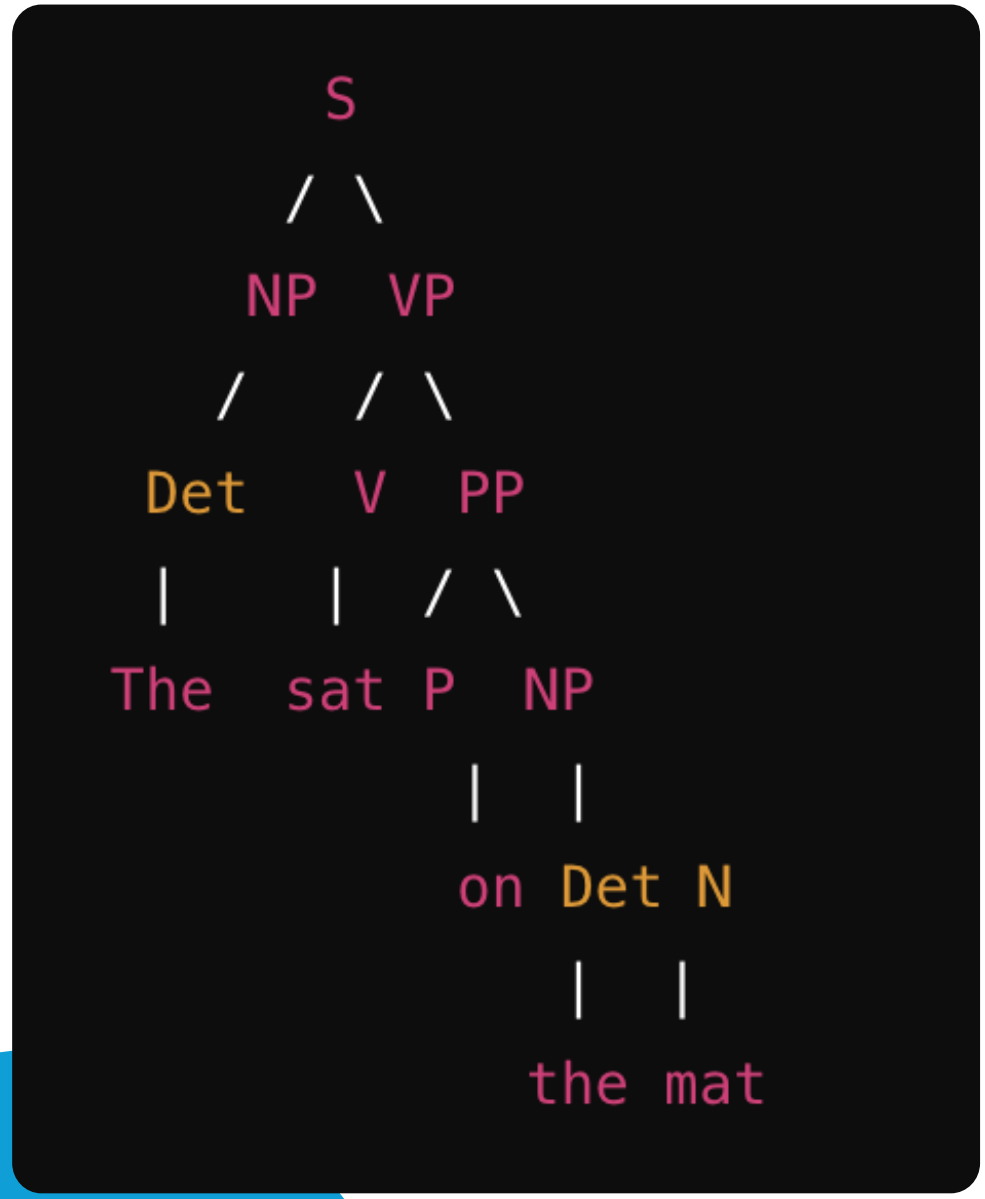
Syntactic and Semantic Processing

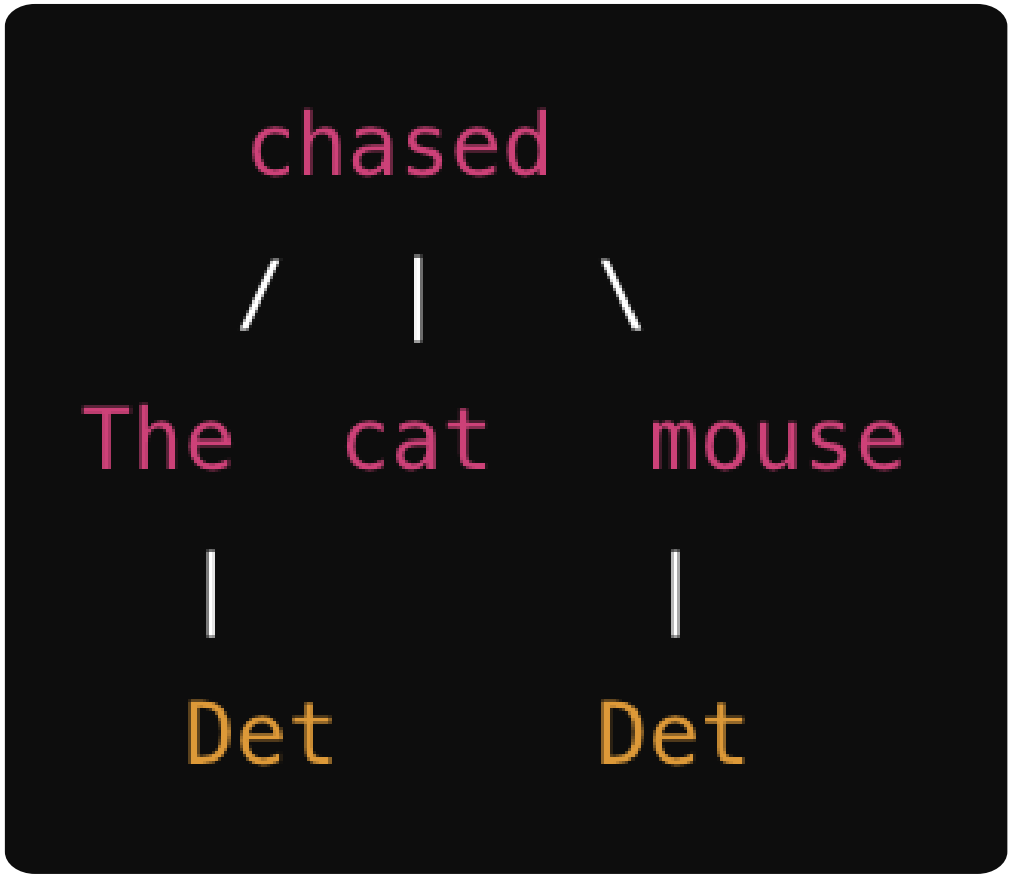
- **Syntactic Processing**: Focuses on the structure and grammatical relationships within a sentence, such as how words are arranged and how they relate to each other (e.g., subject-verb-object relationships).
- **Semantic Processing**: Focuses on the meaning of the words, phrases, and sentences, including the relationships between different concepts and entities within the text.



Parse Trees

- A **parse tree** (or syntax tree) represents the syntactic structure of a sentence according to a formal grammar. It shows how the sentence is derived from the start symbol of the grammar.
- **Example :** Consider the sentence: "The cat sat on the mat."
  - **S**: Sentence
  - **NP**: Noun Phrase
  - **VP**: Verb Phrase
  - **Det**: Determiner
  - **V**: Verb
  - **PP**: Prepositional Phrase
  - **P**: Preposition

Dependency Trees



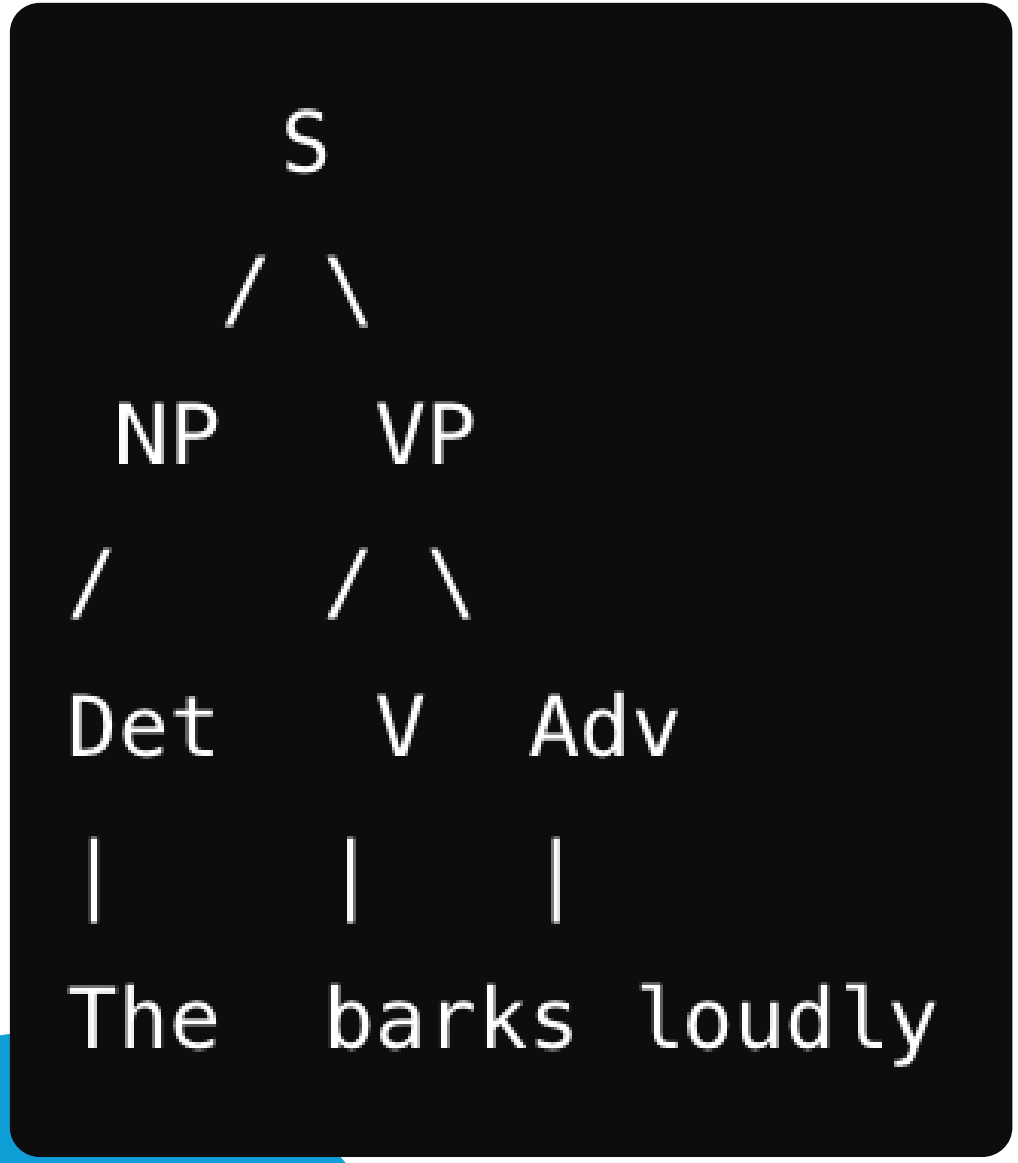
- A **dependency tree** represents the syntactic structure of a sentence by showing how words are connected or depend on each other. Each node in the tree corresponds to a word, and directed edges represent grammatical relations between words.
- **Example:** "The cat chased the mouse."
  - "chased" is the root (the main verb).
  - "The" is dependent on "cat" (determiner dependency).
  - "cat" and "mouse" are dependent on "chased" (subject and object dependencies, respectively).”
  - "The" is dependent on "mouse" (determiner dependency).

Constituency Parsing

- **Constituency parsing** involves breaking down a sentence into its constituent parts (phrases). It creates a parse tree like the first example, where each phrase or sub-phrase is represented as a node.
- Constituency parsing identifies larger units (constituents) that act together as a single unit within a sentence, such as noun phrases (NPs), verb phrases (VPs), and prepositional phrases (PPs).
- Example - "She enjoys playing the piano”
  - **S**: Sentence
  - **NP**: Noun Phrase
  - **VP**: Verb Phrase
  - **V**: Verb
  - **Det**: Determiner

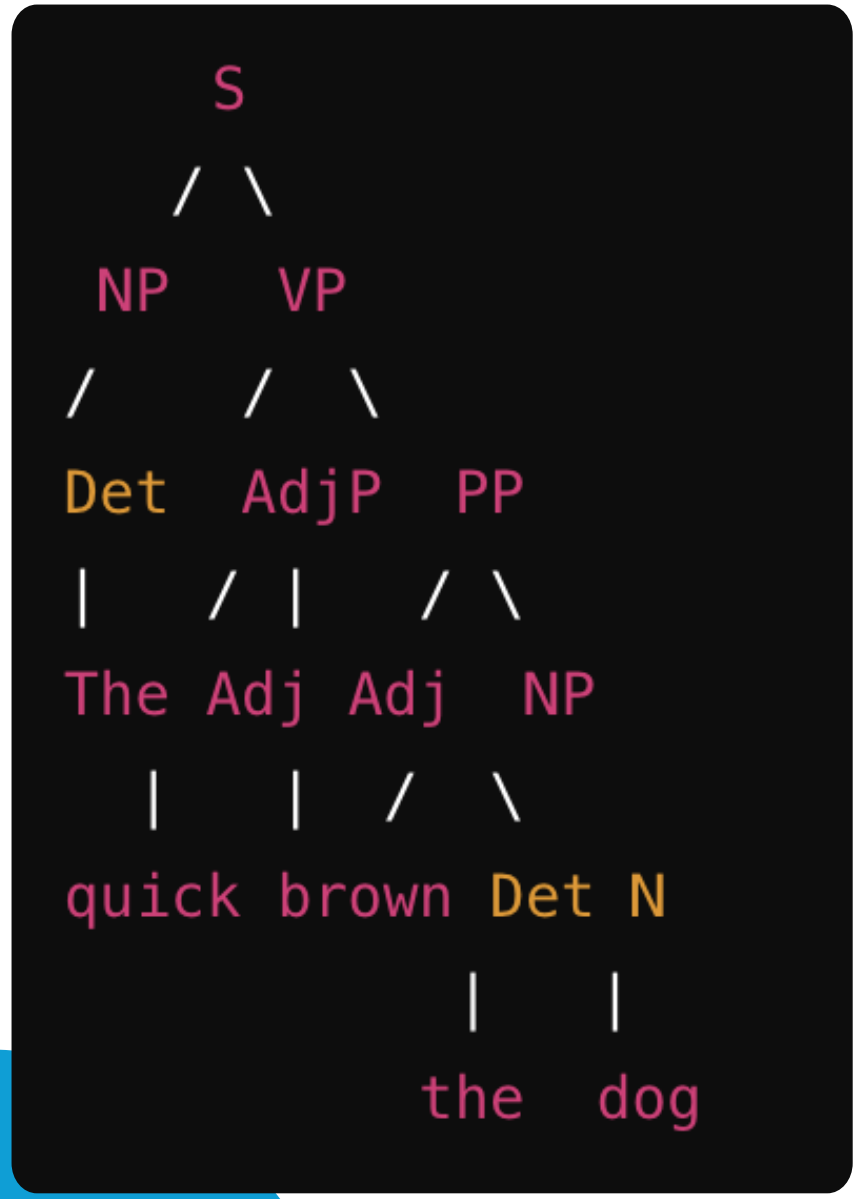
Summary

- **Parse Trees**:
  - Focus on the hierarchical structure of the sentence based on a formal grammar.
  - It show the syntactic structure of a sentence using grammar rules.
- **Dependency Trees**:
  - Emphasize the direct syntactic dependencies between words in the sentence.
  - It display word-to-word relationships based on grammatical dependencies.
- **Constituency Parsing**:
  - Focuses on dividing a sentence into sub-phrases, showing how words group together into constituents.
  - It breaks down a sentence into its constituent phrases, illustrating the hierarchical structure of these phrases.



Common issues - Parse tree construction and interpretation

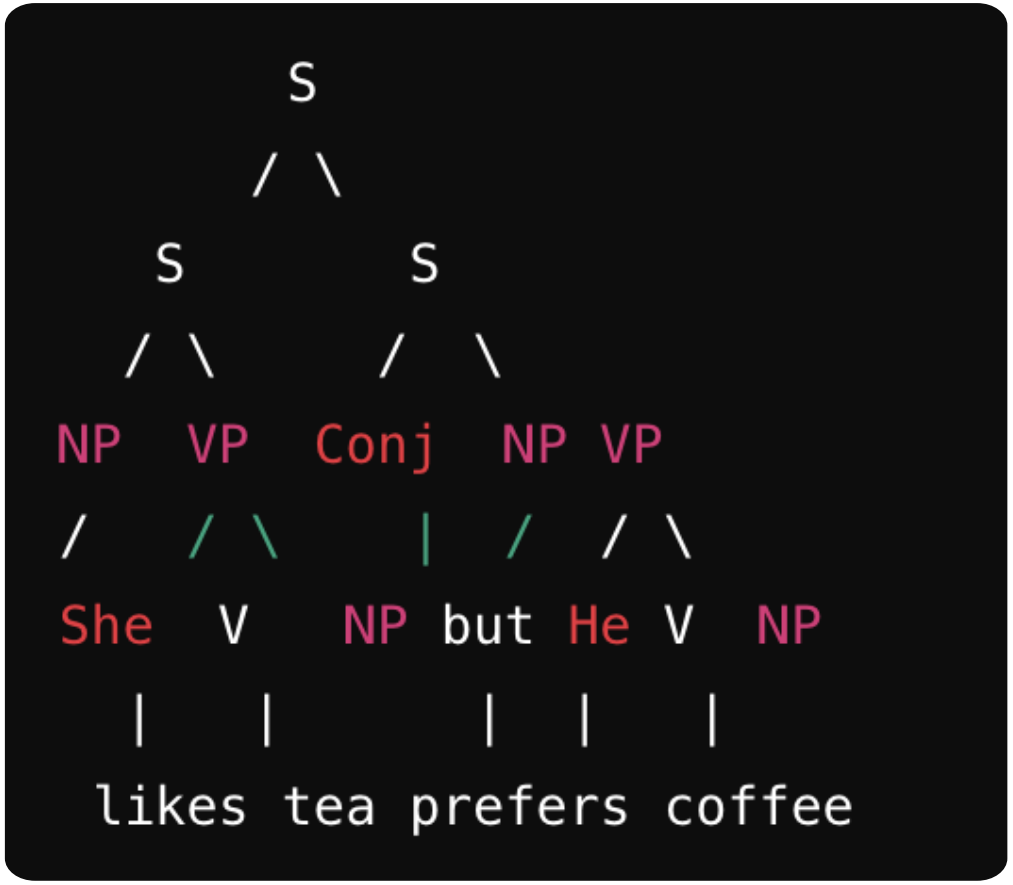
- **1. Identifying the Root of the Parse Tree**
  - **Issue**: Confusion about which part of the sentence should be the root.
  - **Solution**: The root of the parse tree should always be the main verb of the sentence, as it usually represents the main action or state. Everything else in the sentence relates to this verb.
  - **Example**:
    - Sentence: "The dog barks loudly."
    - Root: "barks" (the main verb).



Common issues - Parse tree construction and interpretation

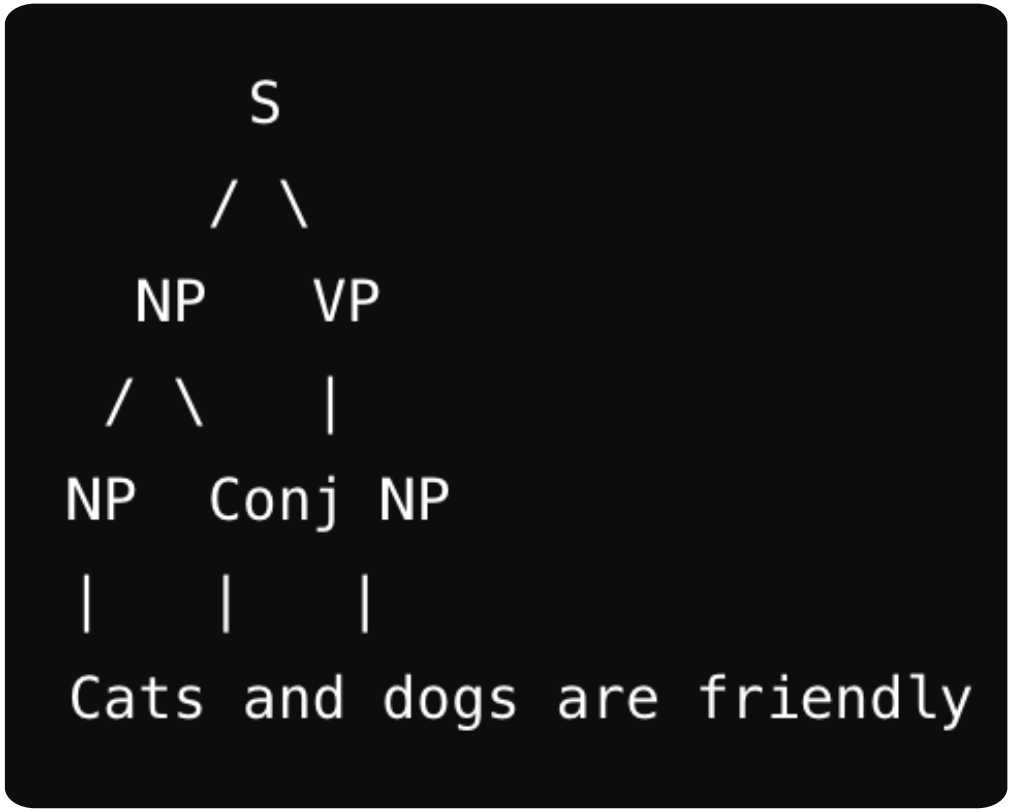
- 2. **Grouping Words into Phrases**
  - **Issue**: Difficulty in identifying how words should be grouped into phrases (e.g., noun phrases, verb phrases).
  - **Solution**: Look for natural groups of words that function together as a single unit. For example, a noun phrase might consist of a determiner and a noun, or a prepositional phrase might consist of a preposition and a noun phrase.
  - **Example**:
    - Sentence: "The quick brown fox jumps over the lazy dog."
    - Grouping:
      - "The quick brown fox" is a noun phrase (NP).
      - "jumps" is a verb phrase (VP).
      - "over the lazy dog" is a prepositional phrase (PP).

- **3. Handling Ambiguity**
  - **Issue**: Sentences can be ambiguous, leading to multiple possible parse trees.
  - **Solution**: Determine the intended meaning of the sentence by considering the context. Then, construct the parse tree that best reflects that meaning.
  - **Example**: "He saw the man with the telescope."
  - Ambiguity:
    - Does "with the telescope" describe "the man" (i.e., the man has a telescope)?
    - Or does it describe how "he" saw the man (i.e., he used a telescope to see the man)?



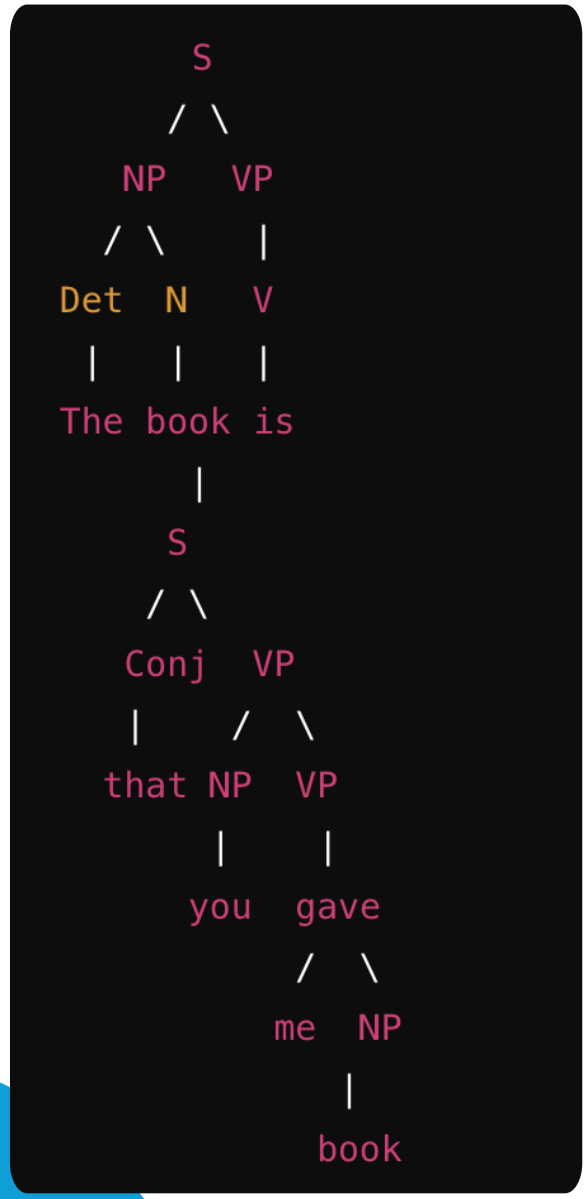
4. Dealing with Compound Sentences

- **Issue**: Struggling with sentences that have more than one clause.
- **Solution**: Treat each clause as a separate sentence with its own structure, and then combine them under a higher-level node.
- **Example**: "She likes tea, but he prefers coffee."
- Clauses:
  - Clause 1: "She likes tea."
  - Clause 2: "He prefers coffee."



5. Understanding Coordination Structures

- **Issue**: Difficulty in parsing sentences with conjunctions like "and" or "or."
- **Solution**: Create a separate node for the conjunction and connect it to the coordinated elements.
- **Example**: "Cats and dogs are friendly."
- Coordination:
  - Cats" and "dogs" are coordinated noun phrases joined by "and."

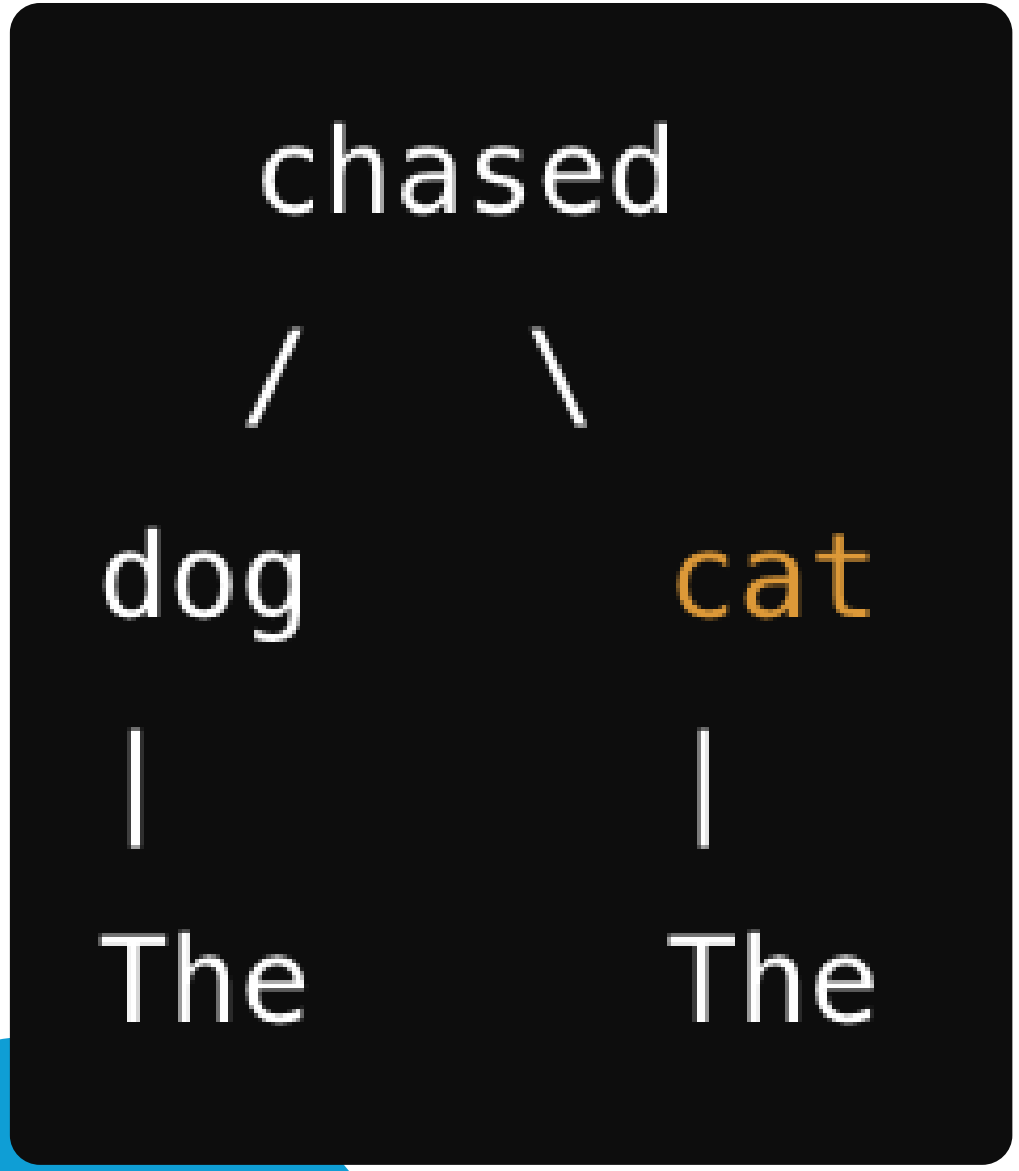


6. Handling Embedded Clauses

- **Issue**: Confusion with sentences containing embedded (subordinate) clauses.
- **Solution**: Treat the embedded clause as a component within the larger sentence structure, typically under a node like NP or VP.
- **Example**: "The book that you gave me is interesting."
- Embedded Clause: "that you gave me"

Key Learnings

- **Identifying the Root**: Always use the main verb as the root of your tree.
- **Grouping Words**: Group words into natural phrases based on their function in the sentence.
- **Handling Ambiguity**: Consider context to choose the correct parse tree.
- **Compound Sentences**: Break them into individual clauses and then combine.
- **Coordination Structures**: Use a conjunction node to connect coordinated elements.
- **Embedded Clauses**: Treat them as part of the larger sentence structure.



Common issues such as dependency parsing algorithms and their applications.

1. Understanding Dependency Relations

- **Issue**: Difficulty in understanding the types of dependencies and how they relate to the sentence structure.
- **Solution**: Learn the basic types of dependency relations, such as subject, object, and modifiers, and understand how they connect words in a sentence.
- **Example**: "The dog chased the cat."
- Dependencies:
  - "dog" → "chased" (subject)
  - "chased" → "cat" (object)
  - "The" → "dog" (determiner)
  - "The" → "cat" (determiner)

2. Selecting a Dependency Parsing Algorithm

- **Issue**: Confusion about which dependency parsing algorithm to use.
- **Solution**: Understand the basic types of dependency parsing algorithms—transition-based, graph-based, and neural network-based—and choose one based on your specific requirements (e.g., accuracy, speed, language support).
- **Transition-Based Parsing**: Builds the dependency tree incrementally, making local decisions at each step. Fast but may suffer from error propagation.
    - Common Algorithm: **Arc-Standard** or **Arc-Eager**.
- **Graph-Based Parsing**: Considers the entire sentence at once and finds the tree that maximizes a global score. More accurate but computationally expensive.
    - Common Algorithm: **Eisner's Algorithm**.
- **Neural Network-Based Parsing**: Uses deep learning to predict dependencies, often yielding state-of-the-art performance.
    - Common Model: **Biaffine Parser**.

3. Handling Non-Projective Structures

- **Issue**: Struggles with sentences that have non-projective (crossing) dependencies, which are challenging for many parsers.
- **Solution**: Use algorithms that can handle non-projective structures, such as graph-based parsers or specific transition-based parsers with techniques like pseudo-projective parsing.
- **Example**: "The book that John read was interesting."
- Non-Projective Dependency:
  - "book" → "read" (object)
  - "read" → "John" (subject)
  - "was" → "interesting" (main verb)
- In a non-projective structure, "read" and "John" would be connected even though other dependencies intervene.
- **Graph-Based Parsing** handles such cases better by considering global information.

4. Error Propagation in Transition-Based Parsers

- **Issue**: Transition-based parsers may propagate errors if an early decision is incorrect.
- **Solution**: Use beam search or lookahead mechanisms to mitigate error propagation by considering multiple possible actions at each step.
- **Example**: "The quick brown fox jumps over the lazy dog."
- If the parser incorrectly assigns "fox" as the subject of "over" early in the process, it could lead to a completely incorrect parse.
- Using **beam search** allows the parser to keep track of multiple hypotheses, reducing the impact of such early errors.

5. Understanding Application of Dependency Parsing

- **Issue**: Uncertainty about how dependency parsing can be applied in real-world scenarios.
- **Solution**: Recognize the wide range of applications, including information extraction, machine translation, sentiment analysis, and question answering.
- **Examples**:
- **Information Extraction**: Dependency parsing can help identify relationships between entities, such as extracting "Person-Organization" pairs in sentences like "John works at Google."
- **Machine Translation**: Dependency structures help in understanding the syntactic roles of words, improving the quality of translations.
- **Sentiment Analysis**: Parsing can identify the relationship between sentiment-bearing words and their targets, enhancing the accuracy of sentiment detection.
- **Question Answering**: Dependency parsing helps understand the structure of questions and answers, allowing for more accurate matching.

6. Dealing with Ambiguity

- **Issue**: Ambiguity in sentences can lead to multiple valid dependency parses.
- **Solution**: Use probabilistic or neural network-based parsers that assign probabilities to different parses, choosing the most likely one.
- **Example**: "He saw the man with the telescope."
- Ambiguity:
  - Is "with the telescope" modifying "saw" or "man"?
- **Probabilistic Parsing**:
  - A probabilistic parser might determine that "with the telescope" is more likely to modify "saw" based on contextual training data.

7. Interpretation of Dependency Trees

- **Issue**: Difficulty in interpreting complex dependency trees.
- **Solution**: Break down the tree into smaller parts, understanding the role of each word and its dependencies.
- **Example**: "The chef who won the award cooks delicious meals."
- Dependency Tree Interpretation:
  - "chef" → "cooks" (subject)
  - "chef" → "won" (relative clause)
  - "won" → "award" (object)
  - "cooks" → "meals" (object)
  - "meals" → "delicious" (adjective)
- By analyzing each part separately, the overall structure becomes clearer.

Key Learnings

- **Understanding Relations**: Focus on basic dependency types and how they structure sentences.
- **Algorithm Selection**: Choose between transition-based, graph-based, or neural network-based algorithms based on your needs.
- **Non-Projective Structures**: Use graph-based parsers for complex non-projective dependencies.
- **Error Propagation**: Mitigate errors in transition-based parsers using techniques like beam search.
- **Applications**: Apply dependency parsing in information extraction, machine translation, sentiment analysis, etc.
- **Ambiguity**: Use probabilistic approaches to handle ambiguous sentences.
- **Tree Interpretation**: Break down complex trees into smaller, understandable parts.

Constituency parsing

- Constituency parsing, also known as phrase structure parsing, is a key task in Natural Language Processing (NLP) that involves breaking down a sentence into its constituent parts (phrases) according to a set of grammar rules. Unlike dependency parsing, which focuses on relationships between individual words, constituency parsing emphasizes the hierarchical structure of sentences, showing how words group together to form phrases and how these phrases function within a sentence.

Common issues in Constituency parsing

- **Understanding the Hierarchical Structure**:
  - Constituency parsing involves understanding the hierarchical nature of language, where sentences are composed of nested phrases (e.g., noun phrases, verb phrases). This can be challenging because it requires recognizing how words group together to form meaningful units.
- **Handling Ambiguity**:
  - Sentences can often be ambiguous, leading to multiple possible constituency parses. Understanding the intended meaning is crucial to selecting the correct parse.
- **Application in NLP**:
  - Constituency parsing is important for various NLP tasks, including machine translation, sentiment analysis, and information extraction, where understanding the structure of sentences is crucial for accurate interpretation.

Constituency Parsing role in NLP

- **Machine Translation**:
  - Constituency parsing helps in understanding the structure of sentences, enabling better translation by preserving the syntactic structure across languages.
- **Sentiment Analysis**:
  - Identifying the structure of sentences can help in determining the scope of sentiment-carrying words, improving the accuracy of sentiment analysis.
- **Information Extraction**:
  - Constituency parsing allows for the extraction of structured information from unstructured text, such as identifying entities and their relationships within a sentence.

Purposes of Semantic Processing

Word Sense Disambiguation (WSD):

- **Purpose**: Identify the correct meaning of a word based on its context. Many words have multiple meanings, and understanding which meaning is intended is crucial.
- **Example**: The word "bank" can refer to a financial institution or the side of a river. Semantic processing helps determine the intended meaning based on context.

Entity Recognition and Linking:

- **Purpose**: Recognize named entities (e.g., people, places, organizations) in text and link them to knowledge bases.
- **Example**: In the sentence "Apple released a new iPhone," semantic processing identifies "Apple" as a company and not the fruit.

Entity Recognition and Linking:

- **Purpose**: Recognize named entities (e.g., people, places, organizations) in text and link them to knowledge bases.
- **Example**: In the sentence "Apple released a new iPhone," semantic processing identifies "Apple" as a company and not the fruit.

Relationship Extraction:

- **Purpose**: Identify relationships between entities within a text.
- **Example**: In "Barack Obama was born in Hawaii," semantic processing extracts the relationship between "Barack Obama" and "Hawaii" as a birth location.

Coreference Resolution:

- **Purpose**: Determine which words refer to the same entity within a text.
- **Example**: In the sentences "John went to the store. He bought milk," semantic processing identifies that "He" refers to "John."

Semantic Role Labeling (SRL):

- **Purpose**: Identify the roles of various constituents in a sentence (e.g., who did what to whom).
- **Example**: In "Sarah gave a book to John," SRL identifies "Sarah" as the giver, "book" as the thing given, and "John" as the receiver.

Textual Entailment:

- **Purpose**: Determine if one sentence logically follows from another.
- **Example**: From "All birds have wings" and "A sparrow is a bird," semantic processing infers that "A sparrow has wings."

Sentiment Analysis:

- **Purpose**: Identify the sentiment or emotional tone of a text.
- **Example**: Semantic processing can determine whether a movie review is positive, negative, or neutral.

Question Answering:

- **Purpose**: Provide accurate answers to questions based on a given text.
- **Example**: For the question "Who is the president of the United States?" semantic processing identifies and extracts the correct answer from the text.